

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Advanced Web Development and Frameworks (ITUE301)

Practical File – ODD Semester 2025–26

Student ID: 24DIT020

Student Name: Gondha Stuti

Practical 5: MongoDB Integration and Schema Design with Mongoose

Problem Definition:

Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.

Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).

Replace the in-memory array from Practical 4 with real Mongoose model operations.

Test all CRUD operations against the live database using Postman.

Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

Code:

Task.js

```
const mongoose = require("mongoose");
```

```
const taskSchema = new mongoose.Schema({

  title:{
    type:String,
    required:true
  },

  description:{
    type:String
  },

  completed:{
    type:Boolean,
    default:false
  },

  createdAt:{
    type:Date,
    default:Date.now
  }

});

module.exports = mongoose.model("Task", taskSchema);
```

server.js:

```
const express = require("express");
const mongoose = require("mongoose");
```

```
const Task = require("./Task");

const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/taskdb")
.then(()=>{

    console.log("MongoDB Connected");

})
.catch((err)=>{

    console.log(err);

});

// GET

app.get("/tasks",async(req,res)=>{

    try{

        const tasks=await Task.find();

        res.status(200).json(tasks);
```

```
}

catch(error){

    res.status(500).json({message:error.message});

}

});

// POST

app.post("/tasks",async(req,res)=>{

    try{

        const task=new Task(req.body);

        const savedTask=await task.save();

        res.status(201).json(savedTask);

    }

    catch(error){

        res.status(400).json({

            error:error.message
```

```
    });

  }

});

// PUT

app.put("/tasks/:id",async(req,res)=>{

  try{

    const updatedTask=await Task.findByIdAndUpdate(

      req.params.id,

      req.body,

      {new:true,runValidators:true}

    );

    if(!updatedTask)

    {

      return res.status(404).json({
```

```
        message:"Task Not Found"

    });

}

res.json(updatedTask);

}

catch(error){

    res.status(400).json({

        error:error.message

    });

}

});

// DELETE

app.delete("/tasks/:id",async(req,res)=>{

    try{

        const task=await Task.findByIdAndDelete(req.params.id);
```

```
    if(!task){

        return res.status(404).json({

            message:"Task Not Found"

        });

    }

    res.json({

        message:"Task Deleted"

    });

}

catch(error){

    res.status(500).json({

        error:error.message

    });

}
```

```
});
```

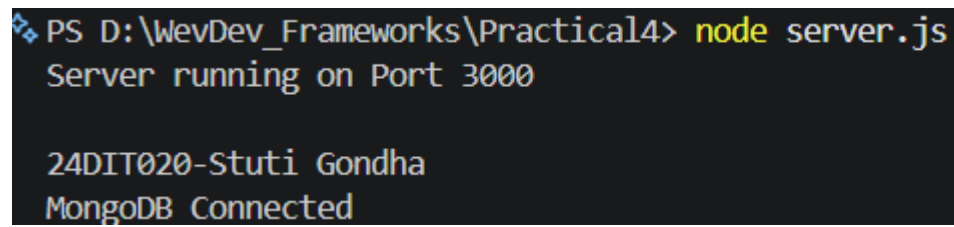
```
app.listen(3000,()=>{
```

```
    console.log("Server running on Port 3000");
```

```
    console.log("\n24DIT020-Stuti Gondha");
```

```
});
```

Output:

A screenshot of a Windows command prompt window. The title bar is partially visible at the top. The command prompt shows the current directory as 'D:\WebDev_Frameworks\Practical4' and the command 'node server.js' has been executed. The output consists of three lines: 'Server running on Port 3000', '24DIT020-Stuti Gondha', and 'MongoDB Connected'.

```
PS D:\WebDev_Frameworks\Practical4> node server.js
Server running on Port 3000

24DIT020-Stuti Gondha
MongoDB Connected
```